

Оптимизация ортогонализации в оптимизаторе Muon

Команда: “Shion Enjoyers”

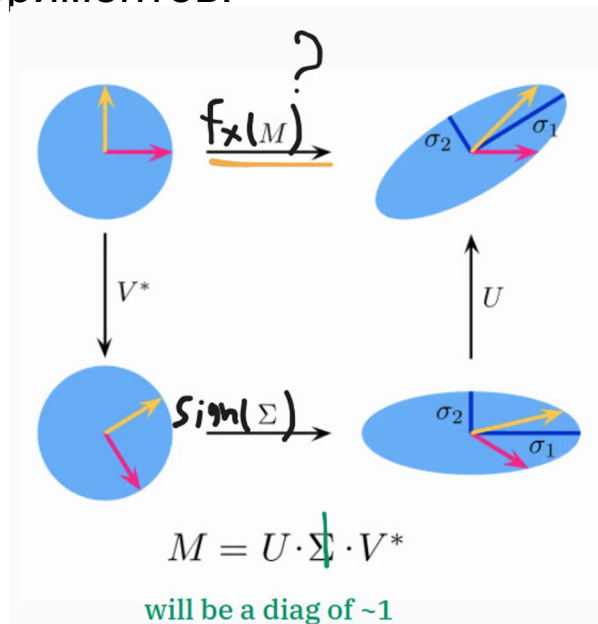
Исупов Даниил
Гусейнов Али

Тема проекта

Обучение больших моделей чувствительно к выбору оптимизатора: он определяет скорость сходимости и стоимость экспериментов.

Оптимизатор MUON:

- Лучше учитывает спектральную структуру градиента
- Потенциально выгодный баланс «качество / вычислительные затраты» по сравнению с AdamW



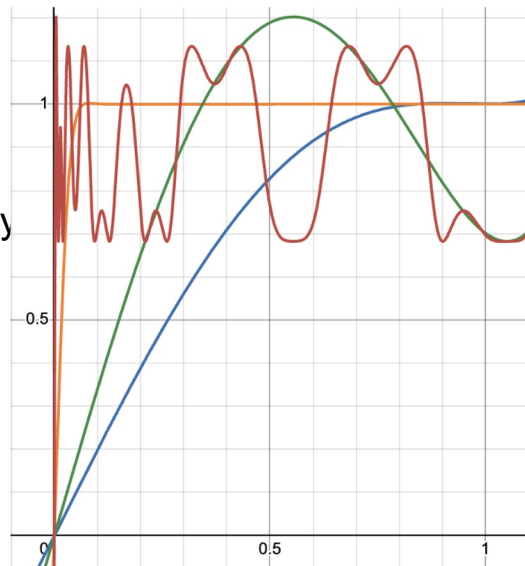
Идея, заложенная в Muon

- Обычные оптимизаторы усиливают уже доминирующие направления в матрице градиента на шагах, слабые почти не двигаются.
- Спектр шага становится сильно разбалансированным, ухудшается обусловленность и стабильность обучения.
- MUON рассматривает градиент как оператор и «выпрямляет» спектр шага, делая изменения по направлениям более равномерными.
- Для этого матричный шаг заменяется на почти ортогональный с помощью быстрой процедуры на базе итерации Ньютона--Шульца

Постановка задачи

Цель проекта: Понять, как сделать MUON практично более эффективным: быстрее обучать модели при сопоставимом или лучшем качестве.

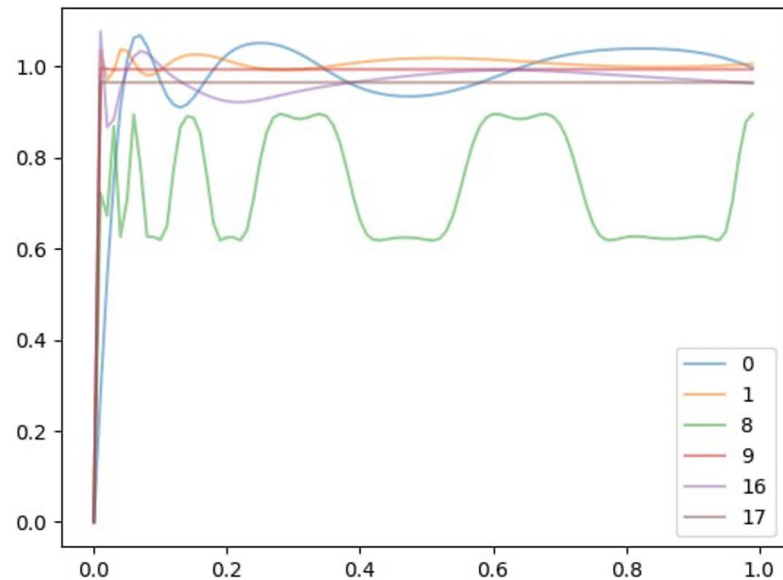
1. Что будет, если изменить аппроксимацию ортогонализации используя другие коэффициенты полинома или полиномы иных степеней?
2. Что будет, если специально настраивать MUON под конкретную задачу?
3. Приведет ли прямая замена Ньютона-Шульца на RSVD к существенному ускорению обучения?
4. Если в реализации RSVD вместо прямого вычисления матрица меньшего ранга будет вычисляться Ньютон-Шульц, ускорит ли это оптимизатор без заметной потери качества?



Эксперимент 1: отбор аппроксимаций НШ

Хотим подобрать семейство скалярных преобразований φ (для сингулярных значений шага MUON), которые хорошо аппроксимируют целевую форму: рассматриваются полиномы, коэффициенты которых ищутся при помощи методов нулевого порядка по функционалу L2:

$$\mathcal{E}(\theta) = \int_0^1 (\text{sign}(x) - p_\theta(x))^2 dx,$$



Эксперимент 1: два способа поиска

- Численный

- Минусы:

- находим полином, который равен единице только в определённых точках

- Сложность вычисления метрики, при k коэффициентах (степень = $2k-1$)

- Сложность по памяти = $O(k)$

- Сложность по времени

- вычисление значение полинома $O(k)$
 - композиция порядка N: $O(\log N)$
 - итоговая сложность $O(nk+n \log N)$

$$\int_0^1 (\text{sign}(x) - p_\theta(x))^2 dx \approx \sum_{i=0}^n (\text{sign}(i/n) - p_\theta(i/n))^2$$

Эксперимент 1: два способа поиска

- Аналитический

$$\int_0^l (\text{sign}(x) - p_\theta(x))^2 dx = \int_0^l (1 - p_\theta(x))^2 dx = l + \sum_{i=0}^{2k} b_{\theta,i} \frac{l^{2i}}{2i}$$

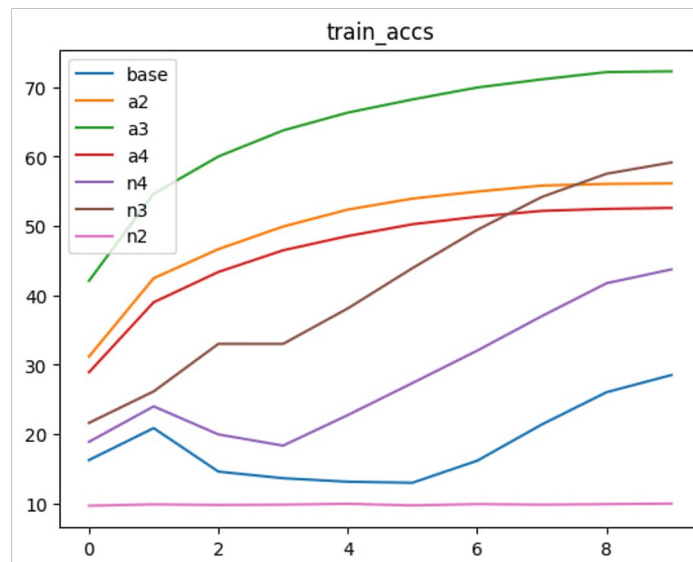
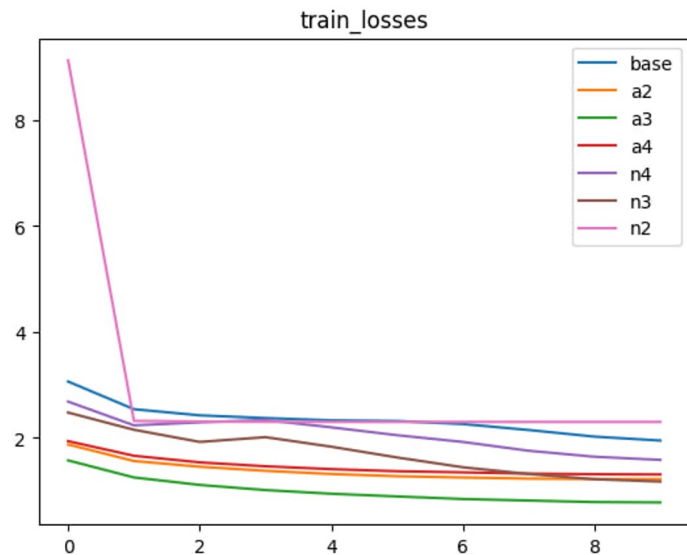
- Минусы:

- коэффициенты при композиции полиномов придётся запоминать
 - если один из коэффициентов стал nan или inf, то полином теряет свою ценность

- Сложность вычисления метрики, при k коэффициентах (степень = 2k-1)

- Сложность по памяти = $O(k^N)$
 - Сложность по времени
 - вычисление значения полинома $O(k^N)$
 - вычислить интеграл $O(k^N)$
 - итоговая сложность $O(k^N)$

Эксперимент 1: валидация на CIFAR10

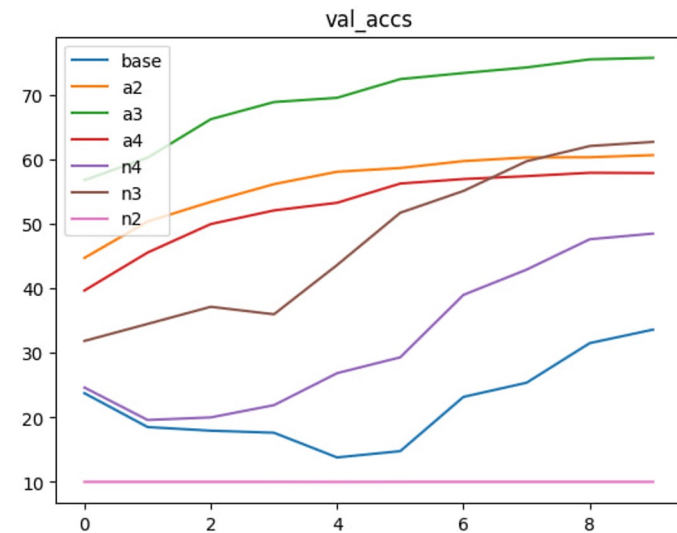
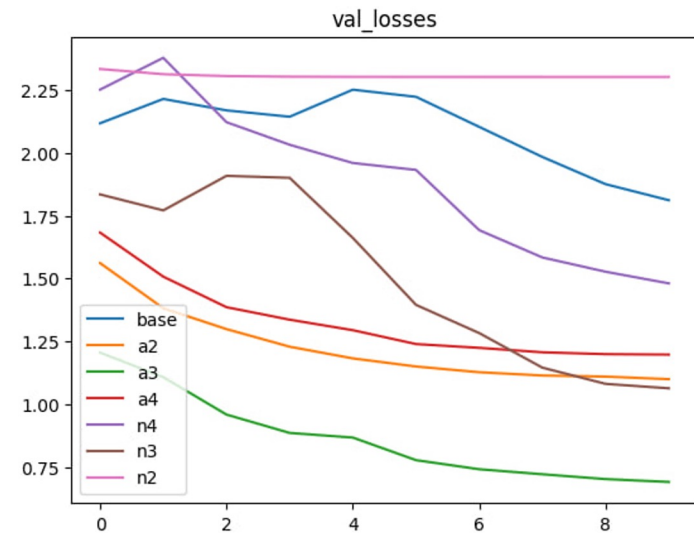


base — стандартные коэффициенты

aN — лучший полином с N коэффициентами, найденный **аналитическим** методом.

nN — лучший полином с N коэффициентами, найденный **численным** методом.

Эксперимент 1: валидация на CIFAR10



base — стандартные коэффициенты

aN — лучший полином с N коэффициентами, найденный **аналитическим** методом.

nN — лучший полином с N коэффициентами, найденный **численным** методом.

Эксперимент 2: Прямая замена Ньютона-Шульца на RSVD

- **Идея:** Используем случайные проекции для эффективного нахождения низкоранговых приближений матриц.
- **Принцип:**
 - Проецирование большой матрицы на случайное подпространство малой размерности.
 - Выполнение стандартных матричных операций (SVD, QR) на полученной *меньшей* матрице.
 - Реконструкция приближённого решения.
- **Преимущества:**
 - Появляется новый гиперпараметр - ранг матрицы, более управляемое обучение и расширение возможностей
 - Значительное сокращение вычислительной сложности: $O(mnk)$ вместо $O(mn\min(m,n))$ для $m \times n$ матрицы и ранга k .
 - Масштабируемость к очень большим матрицам.

Muon: изначальная реализация и улучшение

Algorithm 1 Muon

Require: Learning rate η , momentum μ

```
1: Initialize  $B_0 \leftarrow 0$ 
2: for  $t = 1, \dots$  do
3:   Compute gradient  $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$ 
4:    $B_t \leftarrow \mu B_{t-1} + G_t$ 
5:    $O_t \leftarrow \text{NewtonSchulz5}(B_t)$ 
6:   Update parameters  $\theta_t \leftarrow \theta_{t-1} - \eta O_t$ 
7: end for
8: return  $\theta_t$ 
```

Algorithm 2 MuonWithRSVD

Require: Learning rate η , momentum μ , rank k

```
1: Initialize  $B_0 \leftarrow 0$ 
2: for  $t = 1, \dots$  do
3:   Compute gradient  $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$ 
4:    $B_t \leftarrow \mu B_{t-1} + G_t$ 
5:    $O_t \leftarrow \text{RSVDForMuon}(B_t, k)$ 
6:   Update parameters  $\theta_t \leftarrow \theta_{t-1} - \eta O_t$ 
7: end for
8: return  $\theta_t$ 
```

Эксперимент 3: RSVD с NS

- **Идея:** Сочетает масштабируемость рандомизированных проекций с точностью итераций Ньютона-Шульца для эффективного и качественного нахождения низкоранговых ортогональных направлений обновления градиента.
- **Преимущества:**
 - Сохраняет вычислительные преимущества RSVD ($O(mnk)$) на начальном этапе.
 - Метод Ньютона-Шульца, примененный к компактной матрице B , обеспечивает очень качественную ортогонализацию, избегая затрат на большую матрицу

RSVD: изначальная реализация и улучшение

Algorithm 3 RSVDForMuon

Require: Matrix $A \in \mathbb{R}^{m \times n}$, rank k .

- 1: $\Omega \in \mathbb{R}^{n \times k}$.
 - 2: $Y \leftarrow A\Omega$.
 - 3: $Q \leftarrow \text{QR}(Y)$.
 - 4: $B \leftarrow Q^T A$.
 - 5: $[\tilde{U}, \Sigma, V] \leftarrow \text{SVD}(B)$.
 - 6: $U \leftarrow Q\tilde{U}$.
 - 7: $P \leftarrow UV$
 - 8: **return** P .
-

Algorithm 4 RSVDForMuonWithNS

Require: Matrix $A \in \mathbb{R}^{m \times n}$, rank k .

- 1: $\Omega \in \mathbb{R}^{n \times k}$.
 - 2: $Y \leftarrow A\Omega$.
 - 3: $Q \leftarrow \text{QR}(Y)$.
 - 4: $B \leftarrow Q^T A$.
 - 5: $P \leftarrow \text{NewtonSchulz5}(B)$.
 - 6: **return** P .
-

Метрики качества модели

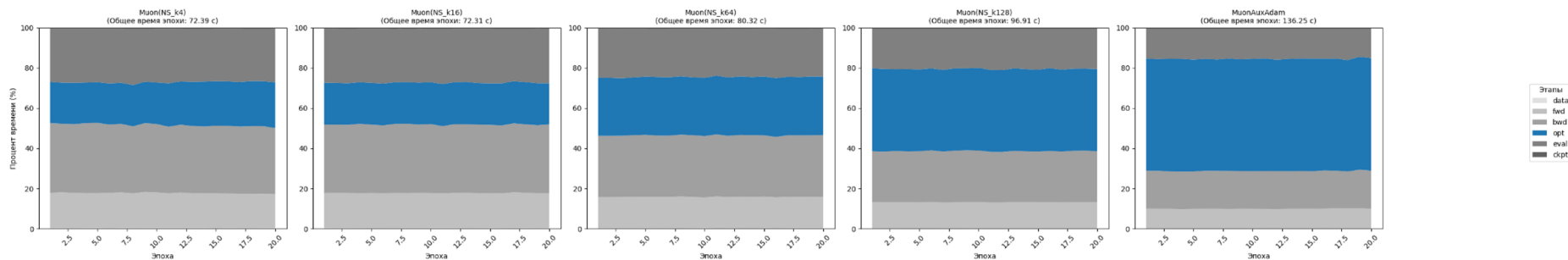
- **Функция потерь (Cross-Entropy):**

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic})$$

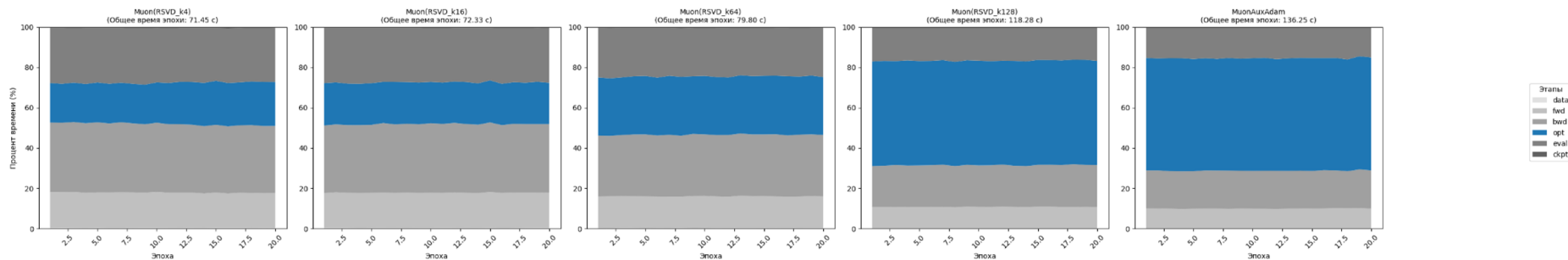
- **Точность (Accuracy):** Доля правильных предсказаний
 - На тренировочной выборке
 - На тестовой выборке
- **Метрики производительности:** Время, затраченное на шаг оптимизатора
- **Датасет:** STL10
- **Что подается на вход оптимизатора:** Матрица, пройденная через сверточную сеть, размером 512x4096, следовательно ранг матрицы как минимум 512

Результаты экспериментов

Распределение времени этапов за эпоху (Muon (Newton-Schulz Low-rank))

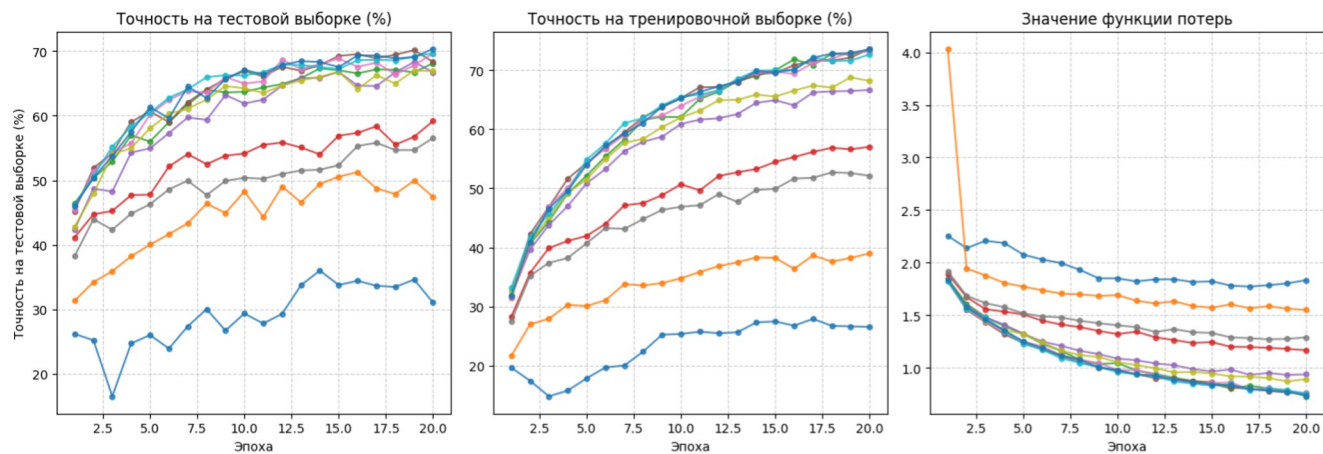


Распределение времени этапов за эпоху (Muon (Randomized SVD Low-rank))



Результаты экспериментов

STL10 Comparison (FAST=False)



Выводы

1. Эксперимент 1:

- Что планировалось: изучить множество коэффициентов
- Что получилось: коэффициенты, которые предложены в изначальной реализации, не универсальны
- Чем можно улучшить: найти метод, который позволяет подбирать коэффициенты под разные задачи

2. Эксперимент 2 и 3:

- Удалось добиться уменьшение скорости в два раза обоими методами, сохранив качество выполнения задачи, как и планировалось
- Проект можно улучшить например тем, чтобы предложить способ нахождения оптимального k для задачи

ОСНОВНЫЕ ССЫЛКИ

- <https://kellerjordan.github.io/posts/muon/#proving-that-ns-iteration-orthogonalizes-the-update>
- <https://github.com/Dont4rootMe/muon-annealing> - github
- <https://docs.pytorch.org/docs/stable/generated/torch.optim.Muon.html> - Muon in pytorch
- <https://colab.research.google.com/drive/1BQSqn8ilxj4ZhInaigyTYRr-XCpLAVor#scrollTo=382c64fb> - ноутбук с реализацией Muon и зависимостью качества и ошибки от коэффициентов